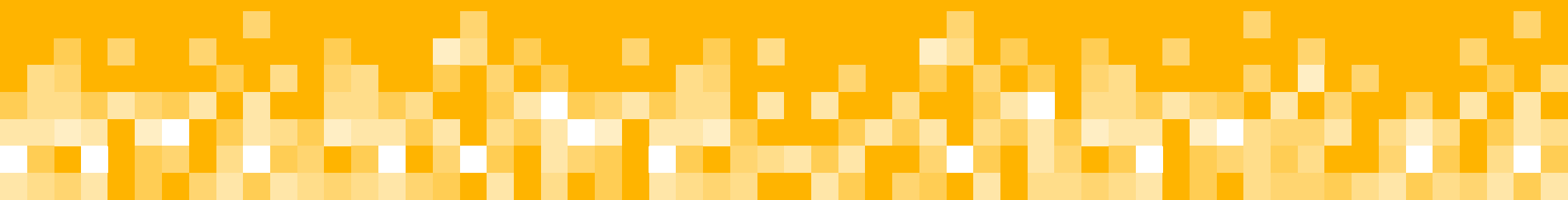


Moduł 3

Refaktoryzacja do wzorca Aggregate

0 regułach biznesowych



0 regułach biznesowych

Podczas sesji event stormingu, konsultacji, szkoleń czy rozmów na temat wymagań biznesowych, można spotkać się z różną interpretacją pojęcia „reguły biznesowej”. Prowadzi to do różnych pomysłów implementacyjnych, co z kolei często powoduje naiwne uproszczenie kodu lub przeciwnie - jego niepotrzebne komplikowanie. W tym tekście postaramy się zidentyfikować, sklasyfikować kilka często spotykanych sytuacji, w których pada słowo „reguła”. Nie będziemy skupiać się na akademickich i analitycznych definicjach, tylko na wypracowaniu schematu, który da Tobie możliwość dobrania odpowiedniego elementu konstrukcyjnego do klasy rozwiązywanego problemu.

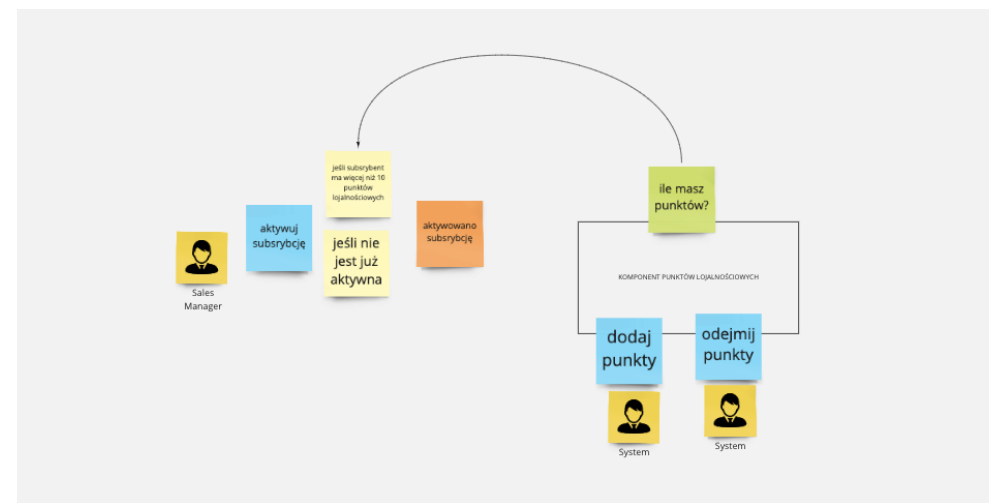
W wielu odcinkach mówiliśmy o regułach biznesowych, których broni nasz system przy obsłudze komendy. Czytelnik w wielu miejscach mógł słyszeć stwierdzenie, że tak zwane **niezmienniki** to reguły biznesowe najczęściej rozumiane jako wyrażenie odnoszące się do stanu obiektu (agregatu), które zawsze muszą być sprawdzone i przestrzegane przez system i ich użytkowników w sposób natychmiastowy. Drugą (i do tej pory ostatnią) grupę reguł często określamy jako te, które system może lekko „rozluźnić” i przykładowo posłużyć się starszymi danymi w celu weryfikacji reguł albo sprawdzić ją w następnej transakcji, idąc ideą spójności końcowej.

Przykładowo:

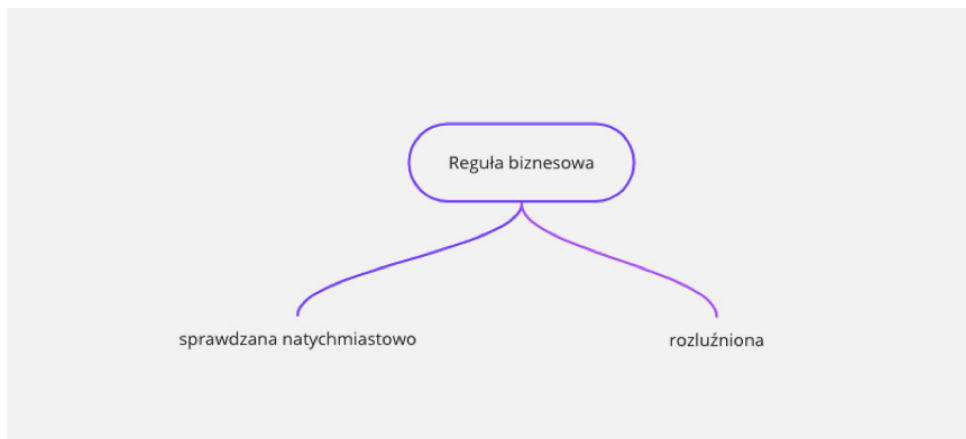
- a) **Niezmiennik**: „Można aktywować subskrypcję **TYLKO** jeśli do tej pory nie jest aktywna”. Sytuację tę przypomniano na poniższym rysunku. Do sprawdzenia czy subskrypcja jest aktywna oraz do jej aktywacji **używamy tej samej danej w systemie**, co oznacza, że zawsze mamy do dyspozycji „najświeższą” informację i dana ta jest optymistycznie lub pesymistycznie blokowana w celu obsługi ewentualnych konfliktów.



- b) **„Rozluźniona” reguła**: „Można pauzować subskrypcję, jeśli mamy więcej niż 10 punktów lojalnościowych”. Jest szansa, że pobraliśmy liczbę punktów, które po pobraniu zostały zmienione przez inną transakcję (klient ma już ich 9). Mogliśmy temu zapobiec, ale z powodów wydajnościowych nie chcemy blokować innych transakcji dotyczących punktów lojalnościowych. Zauważ, że punkty lojalnościowe też mogą mieć swoje niezmienniki (przykładowo: „nie można zejść poniżej zera”), ale najpewniej zajmuje się nimi zupełnie inna część systemu.

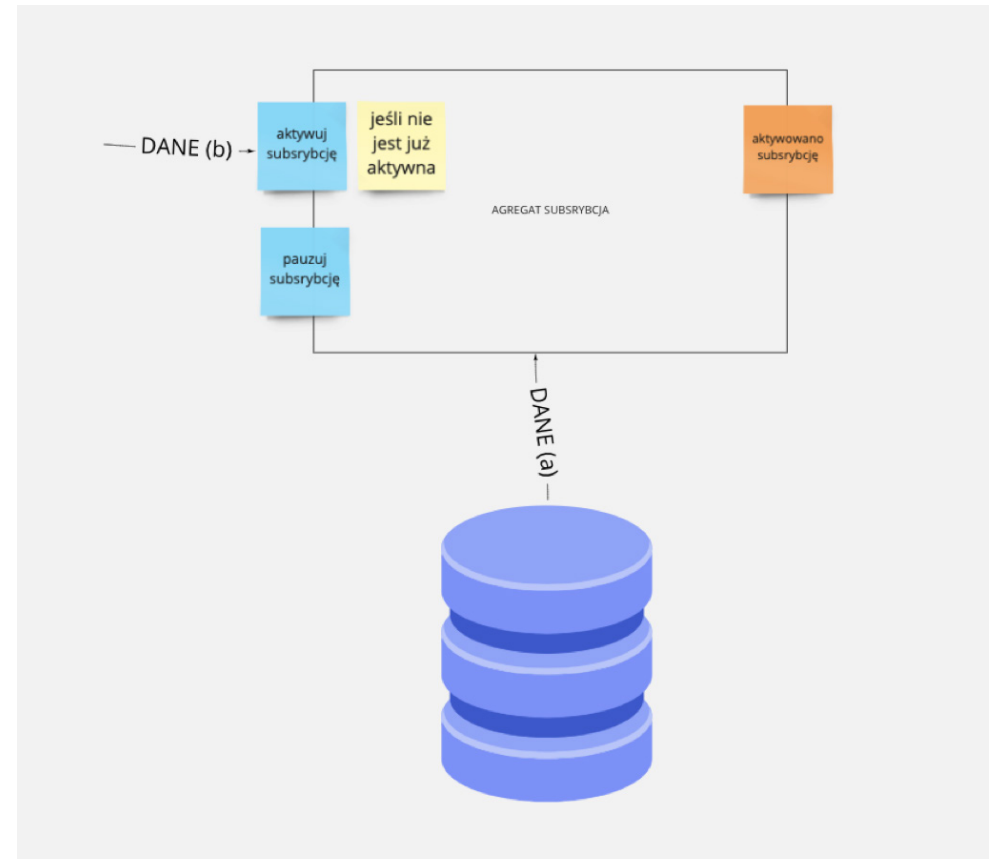


Decyzja czy reguła może być odroczone czy powinna być sprawdzona natychmiast wynika przede wszystkim ze specyfiki problemu biznesowego, ale też narzutów wydajnościowych oraz dostępnej technologii. Zauważ, że powyższy podział reguł jest dokonany poprzez spojrzenie na „świeżość” danych potrzebnych do weryfikacji reguły. Spójrzmy teraz na trochę inną klasyfikację reguł biznesowych.



Zauważ, że aby zapewnić spójność natychmiastową (i ochronić niezmienniki przykładowo dotyczące pauzowania subskrypcji), agregat potrzebuje danych z dwóch źródeł:

- Swoją obecną stan (najpewniej wyciągnięty z bazy danych) - między innymi informacja o tym ile razy subskrypcja była już pauzowana. **Jest to własny stan agregatu, który po pauzowaniu się najprawdopodobniej zmieni (zwiększamy licznik pauz).**
- Informacje, które przychodzą z komendą (na ile chcesz pauzować)



Mając informacje z dwóch źródeł, agregat może podjąć decyzję i zapewnić, że reguła ta nie będzie nigdy złamana, jednocześnie zapewniając obsługę konfliktów wynikających z użytkowników konkurujących do tego zasobu (np. Manager i administrator systemu próbujący pauzować w tym samym momencie).

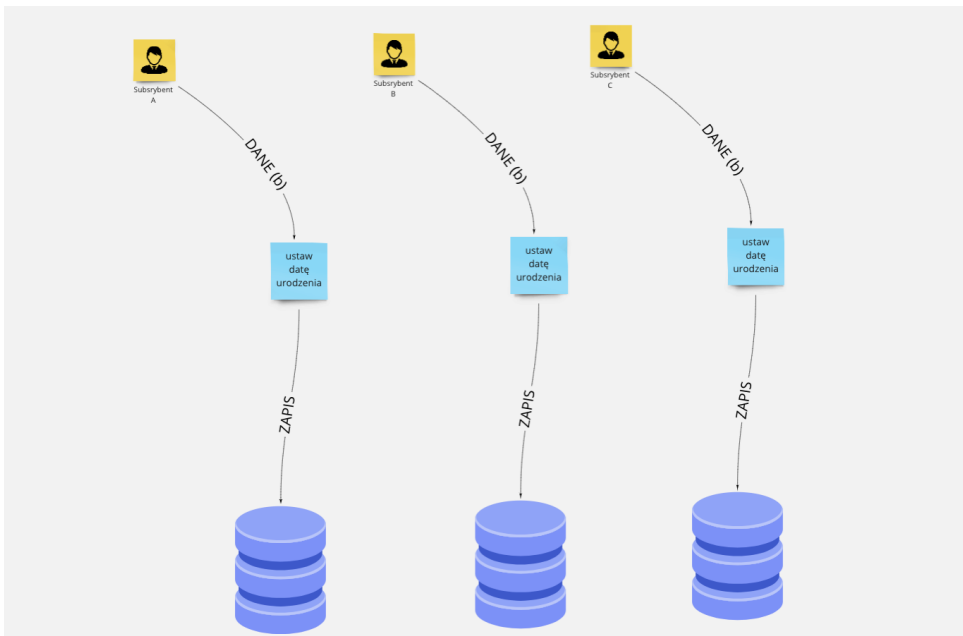
Zauważ też, że **źródłem prawdy o regule jest obecny w naszym systemie agregat**. To on podejmuje decyzje co możemy zrobić z subskrypcją, ma informacje potrzebne do podjęcia tych decyzji i obsługuje komendy (akceptuje lub odrzuca) pochodzące od użytkowników.

Jeśli źródłem prawdy jest właśnie nasz system oraz potrzebujemy danych pochodzących od użytkownika (b) oraz własnej pamięci (a), odpowiednim elementem konstrukcyjnym będzie **agregat**. Wszystkie zasady związane z wydajnym i bezpiecznym projektowaniem agregatów, o których mówiliśmy w tym tygodniu będą miały tutaj zastosowanie.

Często spotykanym problemem jest usilne szukanie miejsca na agregaty (implementowane jako obiekty zapisywane w bazie danych) dla problemów biznesowych, które nie do końca pasują do tego elementu konstrukcyjnego.

Walidacja danych wejściowych

Wyobraźmy sobie następującą **regułę**: „Jeśli użytkownik podał w formularzu zakupu datę urodzenia, to nie może być ona w formacie innym niż DD.MM.YYYY”



Zauważ, że jedyne źródło danych potrzebne do walidacji takiej reguły pochodzi od użytkownika. Nie ma żadnej danej z przeszłości, którą system musi odczytać, aby patrząc na to co pamięta (baza danych) podjąć decyzję. Jedyne co robi, to patrzy na to co otrzymał (od użytkownika). Oznacza to, że natura problemu pozwala nam na stworzenie rozwiązania, które nie będzie miało wąskiego gardła w postaci danych, które należałoby odczytać z bazy na potrzeby sprawdzenia reguły. Nie ma też możliwości równoczesnej próby zmiany tej samej danej przez różnych aktorów. Rozwiązanie może być dużo łatwiej horyzontalnie skalowalne i wydajniejsze. Niektórzy nazywają taką sytuację **walidacją danych wejściowych** albo **lokalnymi niezmiennikami**. Nie szukamy tutaj odpowiednich granic małych agregatów, które pomagają nam zachować spójność, wydajność i nie blokować aktorów wykonujących w tym czasie inne akcje.

Nie oznacza to, że niewskazane tu jest **implementowanie obiektu lub funkcji**, który wszystkie takie dane otrzyma, sprawdzi i zwróci ewentualną listę błędów. Po to, aby przykładowo mieć przypisaną odpowiedzialność i wysoką testowalność. Taki obiekt raczej nie będzie zapisywany w bazie danych, zwraca po prostu informację o tym, że formularz jest poprawny (lub nie), tak aby dalsza część systemu mogła obsłużyć konsekwencję wprowadzonej zmiany. W przypadku w/w systemu, zmiana daty urodzenia może skutkować zwiększeniem punktów lojalnościowych, w przypadku e-commerce, stworzony formularz może posłużyć do utworzenia agregatu zamówienia. Rozmiar obiektu „walidującego” nie będzie miał też znaczenia w kontekście wydajności systemu oraz ewentualnego konfliktu transakcji (bo nie ma z kim konkurować o dane).

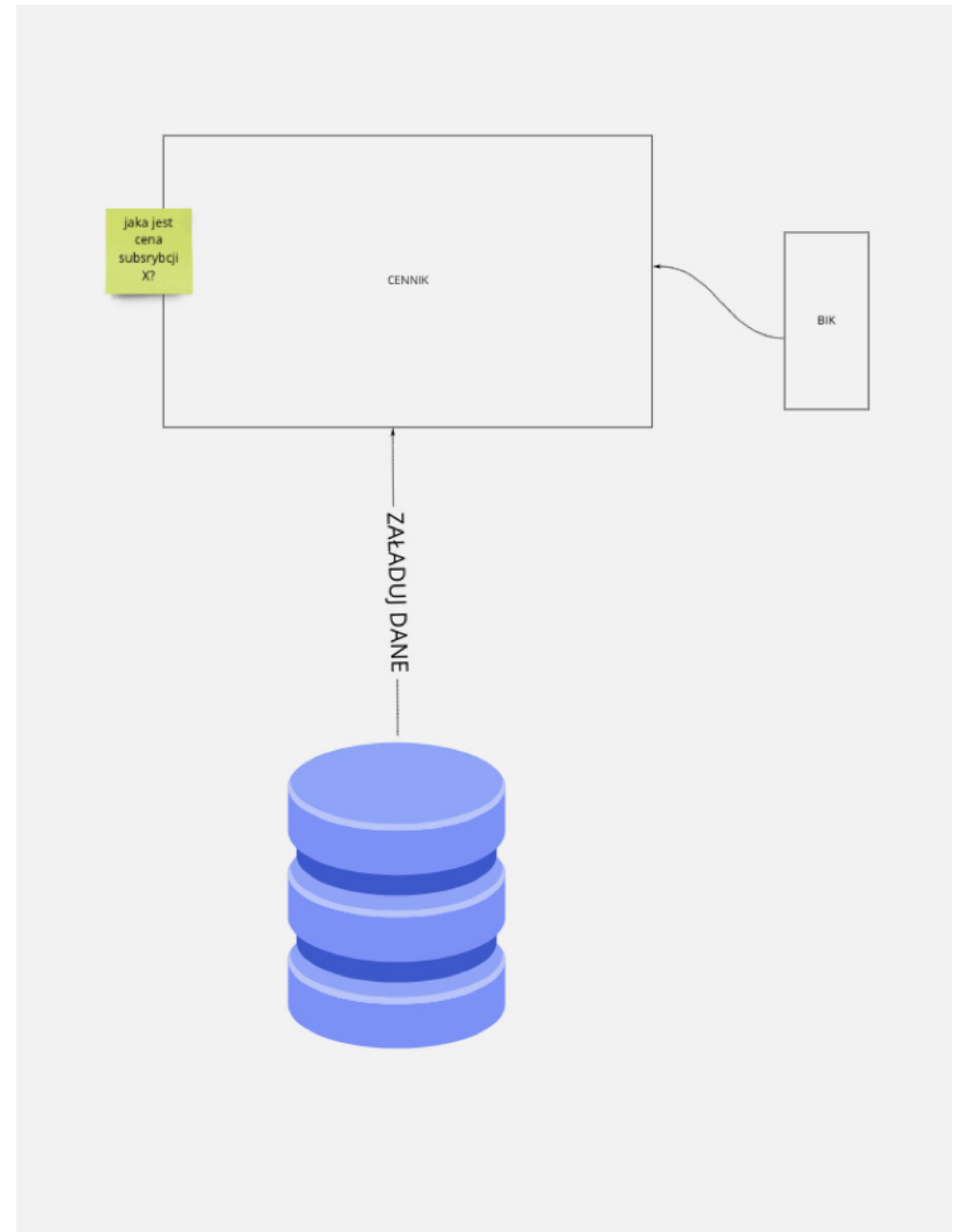
Zauważ, że agregaty, które bronią reguł biznesowych mają dłuższy cykl życia. Tworzymy subskrypcję, w przyszłości ją aktywujemy czy pauzujemy w sposób epizodyczny. W przypadku walidacji, cykl życia obiektu (funkcji) jest krótki, powołujemy go do życia na chwilę i następnie o nim zapominamy.

Obliczenia

Często spotykanym rodzajem reguł są reguły obliczeniowe. Obliczenia mogą być bardzo skomplikowane i zależeć od typu użytkownika, danych, czasu, itp. Wyobraźmy sobie taką **regułę**: „Cena subskrypcji o nazwie X jest obniżana o 10% w środy, o 5 PLN dla klientów VIP oraz jest stała i wynosi 30 PLN jeśli potencjalny subskrybent nie ma złej historii w Biurze Informacji Kredytowej”.

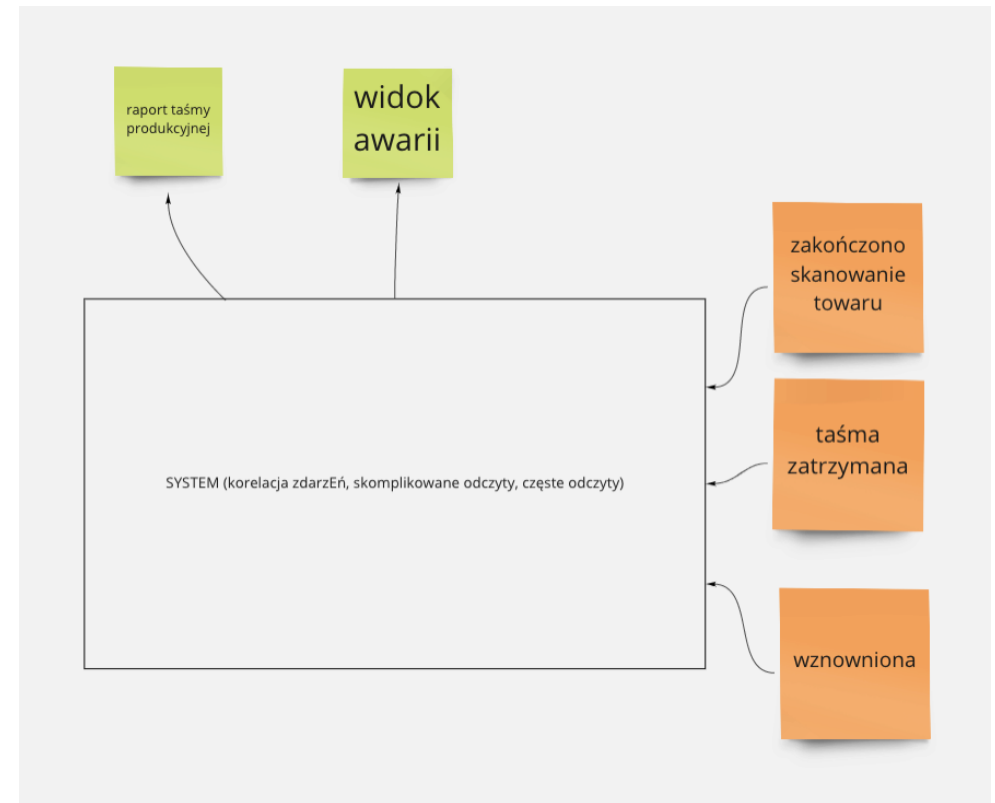
Aby wykonać obliczenia system potrzebuje informacji pochodzącej od użytkowników (nazwa wybranej subskrypcji - czy jest to X), zewnętrznych systemów (BIK), czy też własnego stanu (status klienta oraz dzień tygodnia). Sytuacja wydaje się więc podobna do przypadku z agregatami i pauzowaniem subskrypcji. Jest tu jednak znacząca różnica. Po wykonaniu obliczeń nie zmieniamy stanu systemu w naszej bazie danych (być może zapisujemy informacje o obliczonej cenie tylko w celach audytorskich albo wydajnościowych, nie w celach weryfikacji przyszłych reguł obliczania ceny dla tego samego klienta).

W takim wypadku, dana po prostu zwracana jest do klienta (być może w wiadomości e-mail albo jest wyświetlana na stronie). Implementacja powinna być nastawiona na elastyczność i łatwość dodawania, włączania i wyłączania nowych sposobów obliczeń ceny, nie na szukanie granic agregatów zapewniających spójność przy jednoczesnej wydajności. Podobnie jak w przypadku walidacji, nie oznacza to, że stworzenie obiektu (lub kilku obiektów) odpowiedzialnego za sprawdzenie statusu klienta, dnia tygodnia, policzenie i zwrócenie ceny się nie opłaci, chociażby ze względów na testowalność. Ważne jest, że nie zapisujemy jego stanu po operacji, w celu zapamiętania nowych danych, potrzebnych do walidacji kolejnych komend. Podobnie jak w przypadku obliczeń, cykl życia takiego obiektu jest krótki.



Systemy, które przyjmują tylko zdarzenia

Jest pewna klasa systemów, których jedynym zadaniem jest przyglądanie się zachodzącym w świecie rzeczywistym procesom i zdarzeniom. Mimo, że te systemy mogą być implementacyjnie skomplikowane, mieć dużo linii kodu, warto zauważyć, że nie ma tam miejsca na typowe niezmienniki i agregaty. Dlaczego? Ponieważ systemy te nie są źródłem prawdy na temat problemu biznesowego. Przykładem takiego systemu może być system wizualizujący taśmę produkcyjną w fabryce. Może on generować skomplikowane raporty i wykresy w czasie rzeczywistym na temat produkcji. Może pokazywać dane w różny sposób, zależnie od sytuacji, więc będzie miał instrukcje warunkowe. Mogą zapisywać dane w bazie na potrzeby korelacji zdarzeń albo z pobudek wydajnościowych czy audytorskich. Jednakże, wiedza na temat tego, co „można” zrobić na taśmie jest w głowie operatora produkcji. System nie zabroni człowiekowi położyć na taśmie towaru o zbyt dużej wadze czy wymiarze (a może zabronić pauszować subskrypcję na kursy on-line). System nie zabroni operatorowi puścić na taśmę elementy, które nie powinny tam się znaleźć (ale może o tym poinformować, bo rozumie dwa zdarzenia -> „zaplanowano na godzinę 15 towar X” oraz „o godzinie 15 pojawił się towar Y”). W podobny sposób, system nie zabroni lekarzowi przypisania leku dla pacjenta (być może w momencie wypisywania recepty, lekarz nawet dostępu do systemu nie miał). Taka klasa systemów to **zazwyczaj projekcje na zdarzenia, pochodzące ze świata zewnętrznego**. Do implementacji tego typu problemów, warto skupić się na wydajnych modelach do odczytu, odpowiedniej korelacji zdarzeń, niż na szukaniu niezmienników zgrupowanych w agregaty.



Integracje

Ostatnia klasa reguł to reguły, które powstają na skutek potrzeby integracji wielu systemów informatycznych, często obsługujące długotrwałe procesy biznesowe (przewiezienie paczki, udzielenie kredytu). Przykładem może być następujące zdanie: *“jeśli klient jest klientem Premium, najpierw wniosek o pożyczkę należy wysłać do wybranego banku, potem do systemu transakcyjnego, a w momencie otrzymania pozytywnej odpowiedzi od banku, należy o tym poinformować klienta drogą pocztową”*.

W powyższym zdaniu jest dużo warunków, jednak jedyne za co tworzony komponent będzie odpowiedzialny to odpowiednia obsługa procesu. Być może wygląda ona zupełnie inaczej (jest jeszcze bardziej złożona) dla klientów, którzy nie mają statusu Premium. Mimo złożoności tego problemu, **niezmienniki i agregaty** są najpewniej po stronie banku oraz systemu transakcyjnego, a naszą odpowiedzialnością jest odpowiednie przeprowadzenie procesu biznesowego, więc mamy **reguły procesowe** (kto, kiedy i w jaki sposób jest informowany). Nasz system dostaje informacje o wynikach działań innych systemów oraz wyzwala kolejne kroki procesu, dlatego odpowiednim narzędziem implementacyjnym będzie wspomniany w tym tygodniu **process manager** lub **saga**, a nie agregat.

Wszędzie są “ify”

Jak widać, „if ifowi nie równy” - to, że mamy w naszym systemie instrukcje warunkowe, nie znaczy, że mamy miejsce na agregaty. Pamiętaj też, że zawarta w tekście klasyfikacja nie jest akademickim sposobem generalizacji reguł biznesowych. Skupiliśmy się na problemach i nazwach, które, z naszego doświadczenia pojawia się najczęściej w systemach które widzieliśmy.

